Idéalisation du pouvoir des pays face aux transformations de l'UE

TIPE

Hugo Vangilluwen - 46848

8 juin 2025

CECA

UE

Parlement européen

1. Théorie des indices de pouvoir
2. Système de vote idéal
3. Spectre politique gauche/droite

1. Théorie des indices de pouvoir
2. Système de vote idéal
3. Spectre politique gauche/droite

Coprésident
4 Voix

Coprésident
4 Voix

Secrétaire
1 Voix

Majorité : 5 Voix

Coprésident 4 Voix	Coprésident 4 Voix	Secrétaire 1 Voix

Majorité : 5 Voix

Le secrétaire et les coprésidents influencent autant le scrutin.

Définition 1

Jeu de vote pondéré : Pour / Contre
$\mathcal{V}$ ensemble des votants. Généralement, $\mathcal{V} = [\![1; N]\!]$.

$$(q, (w_i)_{i \in \mathcal{V}}) \in \mathbb{N} \times \mathbb{N}^{\mathcal{V}}$$

Poids total $W = \sum_{i \in \mathcal{V}} w_i$
Quota relatif $q_f \in [0; 1]$ tel que $q = \lceil q_f W \rceil$

$$\mathcal{V} = [\![1; 3]\!]$$
$$q = 5$$
$$w_1 = 4$$
$$w_2 = 3$$
$$w_3 = 2$$
$$W = 9$$
$$q_f = 1/2$$

Définition 1

Jeu de vote pondéré : Pour / Contre
$\mathcal{V}$ ensemble des votants. Généralement, $\mathcal{V} = [\![1; N]\!]$.

$$(q, (w_i)_{i \in \mathcal{V}}) \in \mathbb{N} \times \mathbb{N}^{\mathcal{V}}$$

Poids total $W = \sum_{i \in \mathcal{V}} w_i$
Quota relatif $q_f \in [0; 1]$ tel que $q = \lceil q_f W \rceil$

Définition 2

Coalition gagnante $C \subset \mathcal{V}$

$$\sum_{i \in C} w_i \geqslant q$$

Notons $\mathcal{G} \subset \mathcal{P}(\mathcal{V})$ l'ensemble des coalitions gagnantes.

$$\begin{aligned}
\mathcal{V} &= [\![1; 3]\!] \\
q &= 5 \\
w_1 &= 4 \\
w_2 &= 3 \\
w_3 &= 2 \\
W &= 9 \\
q_f &= 1/2 \\
\mathcal{G} &= \{ \\
&\quad \{1, 2, 3\}, \\
&\quad \{1, 2\}, \\
&\quad \{1, 3\}, \\
&\quad \{2, 3\}, \\
&\quad \{1\} \\
&\}
\end{aligned}$$

Définition 3

Un électeur est dit **décisif** pour une coalition gagnante si le retrait de celui-ci la rend perdante.

Définition 3

Un électeur est dit **décisif** pour une coalition gagnante si le retrait de celui-ci la rend perdante.
La décidabilité d_i pour $i \in \mathcal{V}$ est le nombre de coalitions où i est décisif :

$$d_i = |\{C \in \mathcal{G} \mid i \in C \text{ et } C \setminus \{i\} \notin \mathcal{G}\}|$$

Définition 3

Un électeur est dit **décisif** pour une coalition gagnante si le retrait de celui-ci la rend perdante.
La décidabilité d_i pour $i \in \mathcal{V}$ est le nombre de coalitions où i est décisif :

$$d_i = |\{C \in \mathcal{G} \mid i \in C \text{ et } C \setminus \{i\} \notin \mathcal{G}\}|$$

Le votant 1 est décisif pour $\{1, 2\}$, $\{1, 3\}$ et $\{1\}$.

$$d_1 = 3 \quad d_2 = d_3 = 1$$

Indice de Banzhaf

Définition 4

Indice de Banzhaf normalisé :

$$B_i = \frac{d_i}{\sum_{j=1}^{N} d_j}$$

Indice de Banzhaf absolu :

$$B'_i = \frac{d_i}{2^{N-1}}$$

Indice de Banzhaf

Définition 4

Indice de Banzhaf normalisé :

$$B_i = \frac{d_i}{\sum_{j=1}^{N} d_j}$$

Indice de Banzhaf absolu :

$$B'_i = \frac{d_i}{2^{N-1}}$$

$$B_1 = 3/5 \quad B_2 = B_3 = 1/5$$
$$B'_1 = 3/4 \quad B'_2 = B'_3 = 1/4$$

Algorithme naïf

Complexité : $\mathcal{O}(N2^N)$

Nombre de pays	Temps de calcul
18	0,7 s
19	1,9 s
20	4,4 s
21	9,5 s
22	20,4 s
23	43,7 s

Polynôme générateur de Brams-Affuso

d_i : décidabilité du votant i

c_k^i : nombre de coalitions n'incluant pas i et de poids total k

Polynôme générateur de Brams-Affuso

d_i : décidabilité du votant i

c_k^i : nombre de coalitions n'incluant pas i et de poids total k

$$\begin{cases} i \in C \\ C \in \mathcal{G} \\ C \setminus \{i\} \notin \mathcal{G} \end{cases} \iff \begin{cases} i \in C \\ \sum_{j \in C} w_j = \sum_{j \in C \setminus \{i\}} w_j + w_i \geqslant q \\ \sum_{j \in C \setminus \{i\}} w_j < q \end{cases}$$

Polynôme générateur de Brams-Affuso

d_i : décidabilité du votant i

c_k^i : nombre de coalitions n'incluant pas i et de poids total k

$$\begin{cases} i \in C \\ C \in \mathcal{G} \\ C \setminus \{i\} \notin \mathcal{G} \end{cases} \iff \begin{cases} i \in C \\ \sum_{j \in C} w_j = \sum_{j \in C \setminus \{i\}} w_j + w_i \geqslant q \\ \sum_{j \in C \setminus \{i\}} w_j < q \end{cases}$$

$$\iff \begin{cases} i \in C \\ q - w_i \leqslant \sum_{j \in C \setminus \{i\}} w_j \leqslant q - 1 \end{cases}$$

Polynôme générateur de Brams-Affuso

d_i : décidabilité du votant i
c_k^i : nombre de coalitions n'incluant pas i et de poids total k

$$\begin{cases} i \in C \\ C \in \mathcal{G} \\ C \setminus \{i\} \notin \mathcal{G} \end{cases} \iff \begin{cases} i \in C \\ \sum_{j \in C} w_j = \sum_{j \in C \setminus \{i\}} w_j + w_i \geqslant q \\ \sum_{j \in C \setminus \{i\}} w_j < q \end{cases}$$

$$\iff \begin{cases} i \in C \\ q - w_i \leqslant \sum_{j \in C \setminus \{i\}} w_j \leqslant q - 1 \end{cases}$$

$$d_i = \sum_{k=q-w_i}^{q-1} c_k^i$$

Polynôme générateur de Brams-Affuso

$$\prod_{j \in \mathcal{V},\, j \neq i} (1 + X^{w_j}) \stackrel{?}{=} \sum_{k=0}^{W - w_i} c_k^i X^k$$

Polynôme générateur de Brams-Affuso

$$\prod_{j\in\mathcal{V},\, j\neq i} (1+X^{w_j}) \stackrel{?}{=} \sum_{k=0}^{W-w_i} c_k^i X^k$$

$$\prod_{\substack{j\in\mathcal{V}\\ j\neq i}} (1+1.X^{w_j}) = \sum_{k=0}^{W-w_i} \left(\sum_{\substack{C\in\mathcal{P}(\mathcal{V}\setminus\{i\})\\ \sum_{j\in C} w_j = k}} 1 \right) X^k$$

Polynôme générateur de Brams-Affuso

$$\prod_{j \in \mathcal{V},\, j \neq i} (1 + X^{w_j}) = \sum_{k=0}^{W - w_i} c_k^i X^k$$

$$\prod_{\substack{j \in \mathcal{V} \\ j \neq i}} (1 + 1.X^{w_j}) = \sum_{k=0}^{W - w_i} \left(\sum_{\substack{C \in \mathcal{P}(\mathcal{V} \setminus \{i\}) \\ \sum_{j \in C} w_j = k}} 1 \right) X^k$$

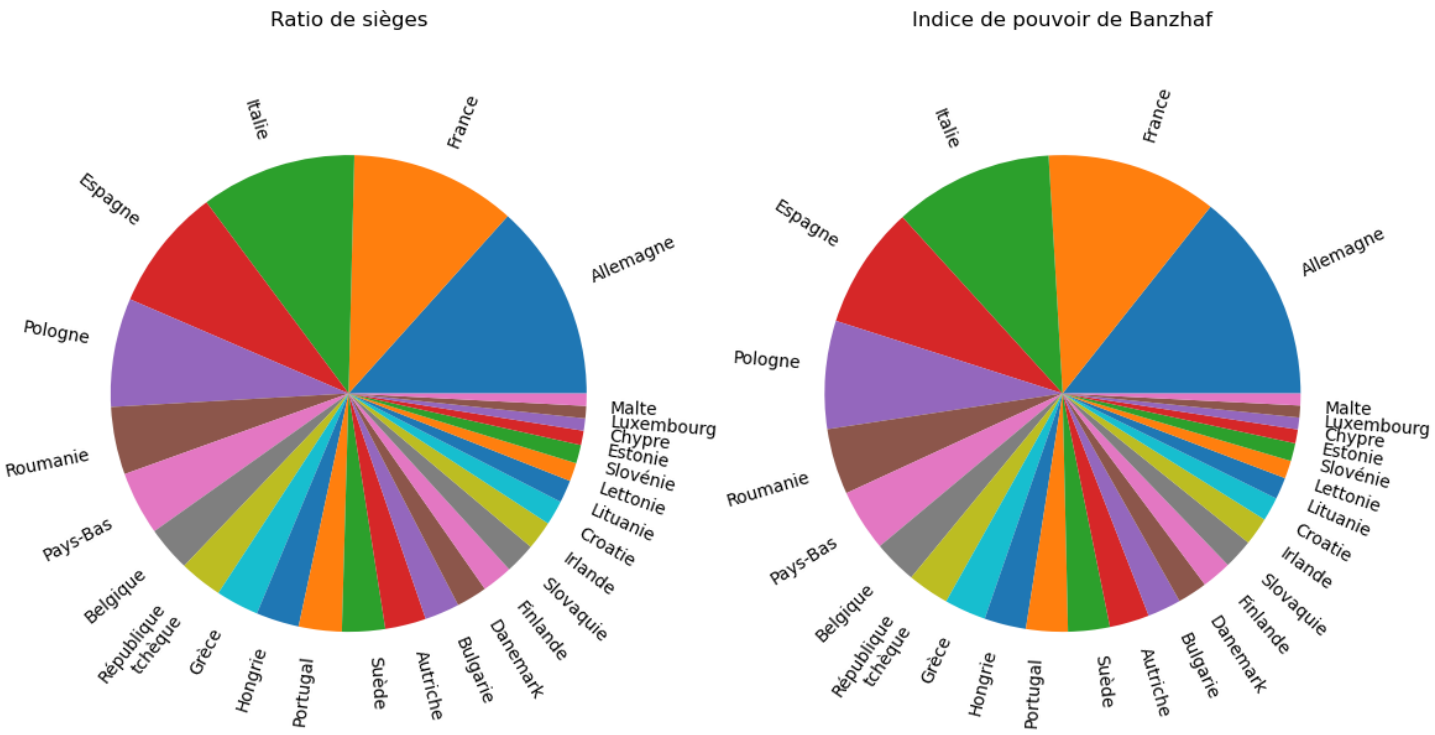
Complexité : $\mathcal{O}(N.W^2)$

1 Théorie des indices de pouvoir

2 Système de vote idéal

3 Spectre politique gauche/droite

Système actuel



Allemagne +7,3% Danemark -3,2%

Algorithme de Leech

$$p_0 = d \in \mathbb{R}^{\mathcal{V}}$$

Classiquement : $p_{n+1} = p_n + \lambda(d - B'(p_n)) \in \mathbb{R}^{\mathcal{V}}$

Algorithme de Leech

$$p_0 = d \in \mathbb{R}^{\mathcal{V}}$$

Classiquement : $p_{n+1} = p_n + \lambda(d - B'(p_n)) \in \mathbb{R}^{\mathcal{V}}$

$$p_{n+1} = p_n + \lambda(n)(d - B'(p_n)) \in \mathbb{R}^{\mathcal{V}}$$

$$\lambda(n) = \frac{\Lambda q}{f(n)} \text{ avec } f(n) = \ln(n) + 1 \text{ et } \Lambda = 0{,}83$$

q = 400

Allemagne	64	96	Bulgarie	16	17
France	55	81	Danemark	15	15
Italie	50	76	Finlande	15	15
Espagne	45	60	Slovaquie	14	15
Pologne	38	53	Irlande	14	14
Roumanie	27	33	Croatie	12	12
Pays-Bas	26	31	Lituanie	11	11
Belgique	21	22	Slovénie	9	9
République tchèque	20	21	Lettonie	8	9
Portugal	20	21	Estonie	7	7
Suède	20	21	Chypre	6	6
Grèce	20	21	Luxembourg	6	6
Hongrie	19	21	Malte	5	6
Autriche	19	20	TOTAL	582	720

Jeu de vote égalitaire

Si $w = (1)_{i \in \mathcal{V}}$ et $q_f = 1/2$ alors

Jeu de vote égalitaire

Si $w = (1)_{i \in \mathcal{V}}$ et $q_f = 1/2$ alors

$$\forall i \in \mathcal{V},\ d_i = \binom{N-1}{\lfloor N/2 \rfloor}$$

Jeu de vote égalitaire

Si $w = (1)_{i \in \mathcal{V}}$ et $q_f = 1/2$ alors

$$\forall i \in \mathcal{V},\ d_i = \binom{N-1}{\lfloor N/2 \rfloor}$$

$$\forall i \in \mathcal{V},\ B_i = \frac{1}{N} \text{ et } B'_i \underset{N \to +\infty}{\sim} 2\sqrt{\frac{2}{\pi}} \frac{1}{\sqrt{N}}$$

Théorème de Penrose

Théorème 5

Tous les participants d'un jeu en 2 étapes ont le même pouvoir si les indices de pouvoir des groupes de la 2ème étape sont proportionnels aux racines carrées de leur cardinal.

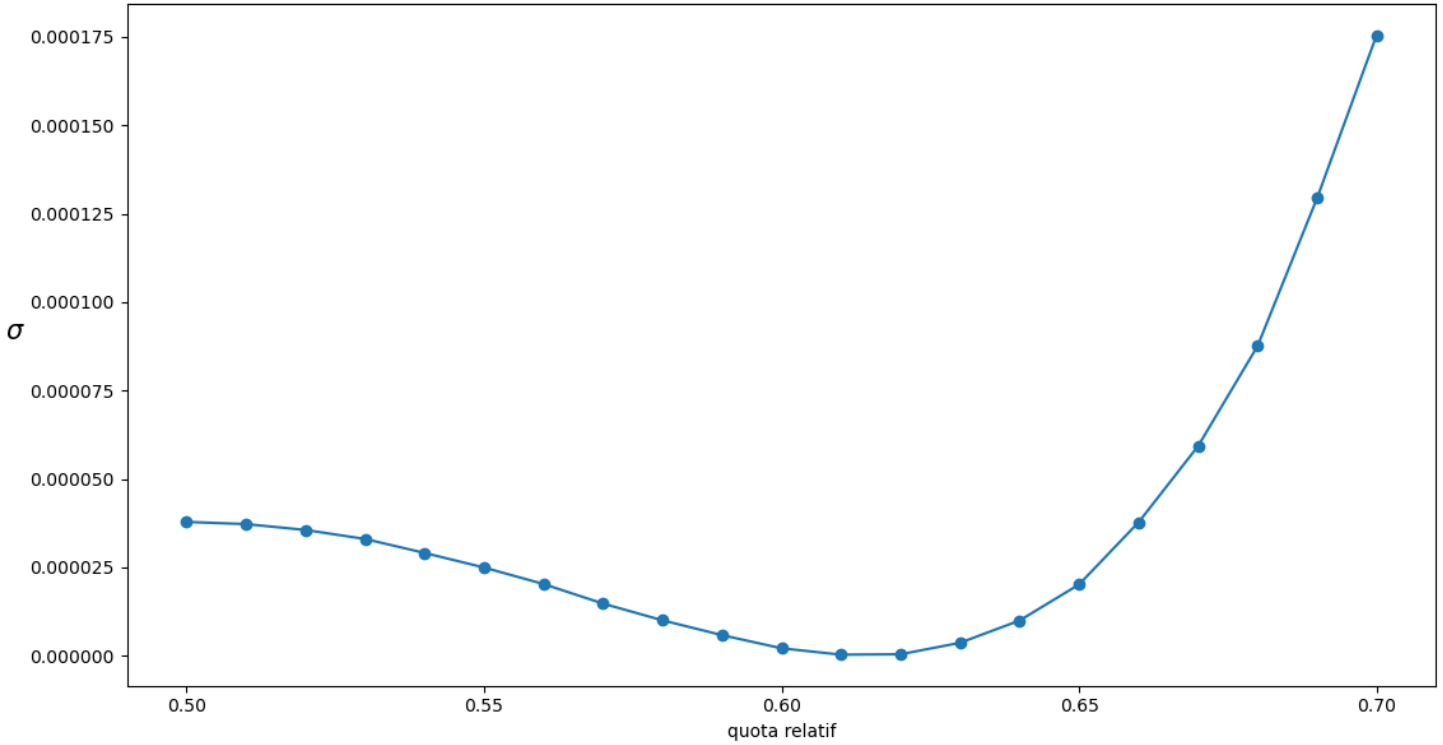
Théorème de Penrose

Théorème 5

Tous les participants d'un jeu en 2 étapes ont le même pouvoir si les indices de pouvoir des groupes de la 2ème étape sont proportionnels aux racines carrées de leur cardinal.

Pour l'UE : tous les citoyens possèdent le même pouvoir si les indices de pouvoirs des pays au Parlement Européen sont proportionnels à la racine carrée de leur population.

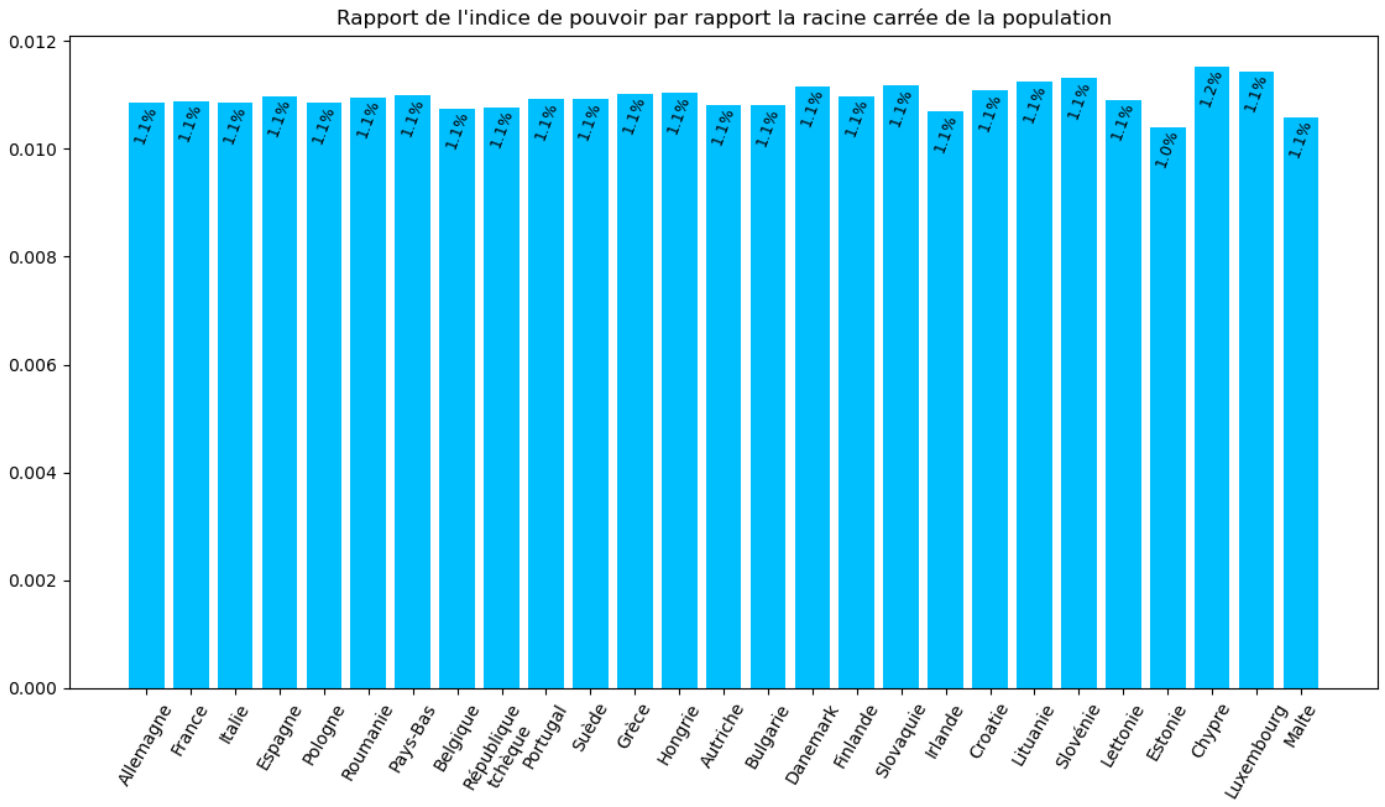
$$\sigma = \sum_{i \in \mathcal{V}} \left(\frac{w_i}{W} - B_i \right)^2$$



$$q = 453$$

Allemagne	73	96	Bulgarie	20	17
France	66	81	Danemark	20	15
Italie	61	76	Finlande	19	15
Espagne	56	60	Slovaquie	19	15
Pologne	48	53	Irlande	18	14
Roumanie	35	33	Croatie	16	12
Pays-Bas	34	31	Lituanie	14	11
Belgique	27	22	Slovénie	12	9
République tchèque	26	21	Lettonie	11	9
Portugal	26	21	Estonie	9	7
Suède	26	21	Chypre	8	6
Grèce	26	21	Luxembourg	7	6
Hongrie	25	21	Malte	6	6
Autriche	24	20	TOTAL	732	720

$$(62\%, (\sqrt{population})_{pays})$$

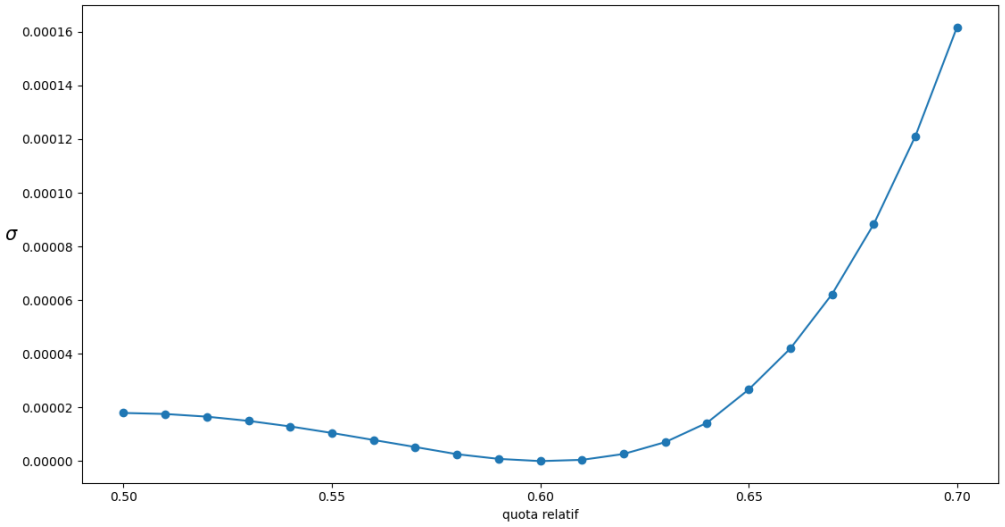


Transformation ultime : "Europe des 36"

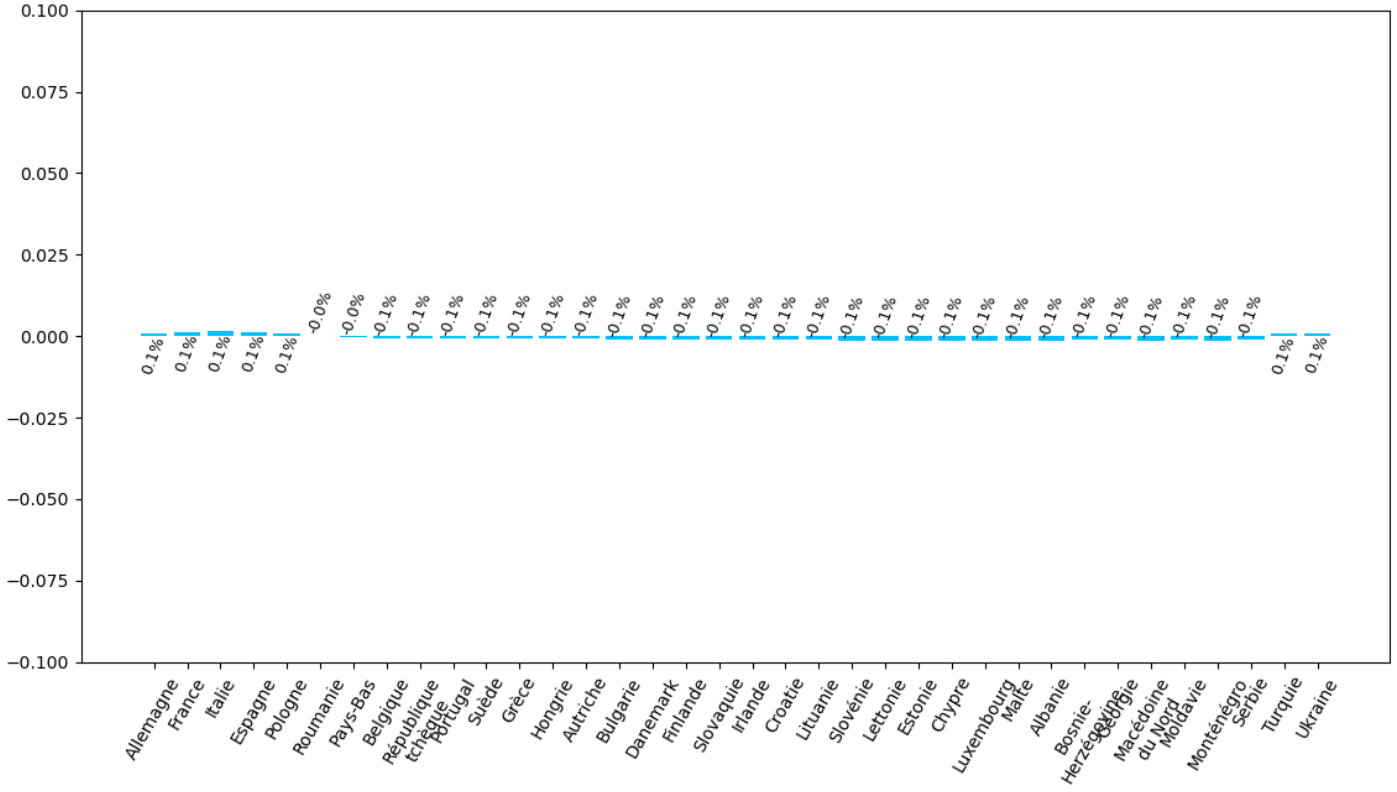
Pays officiellement candidats à l'adhésion à l'Union Européenne

- Albanie
- Bosnie-Herzégovine
- Géorgie
- Macédoine du Nord
- Moldavie
- Monténégro
- Serbie
- Turquie
- Ukraine

$$q_f = 60\%$$



Écart relatif entre le pouvoir et la racine carrée de la population



1. Théorie des indices de pouvoir
2. Système de vote idéal
3. Spectre politique gauche/droite

Coalitions réalistes

$$(\mathcal{V}, \preccurlyeq)$$

Coalitions réalistes

$$(\mathcal{V}, \preccurlyeq)$$

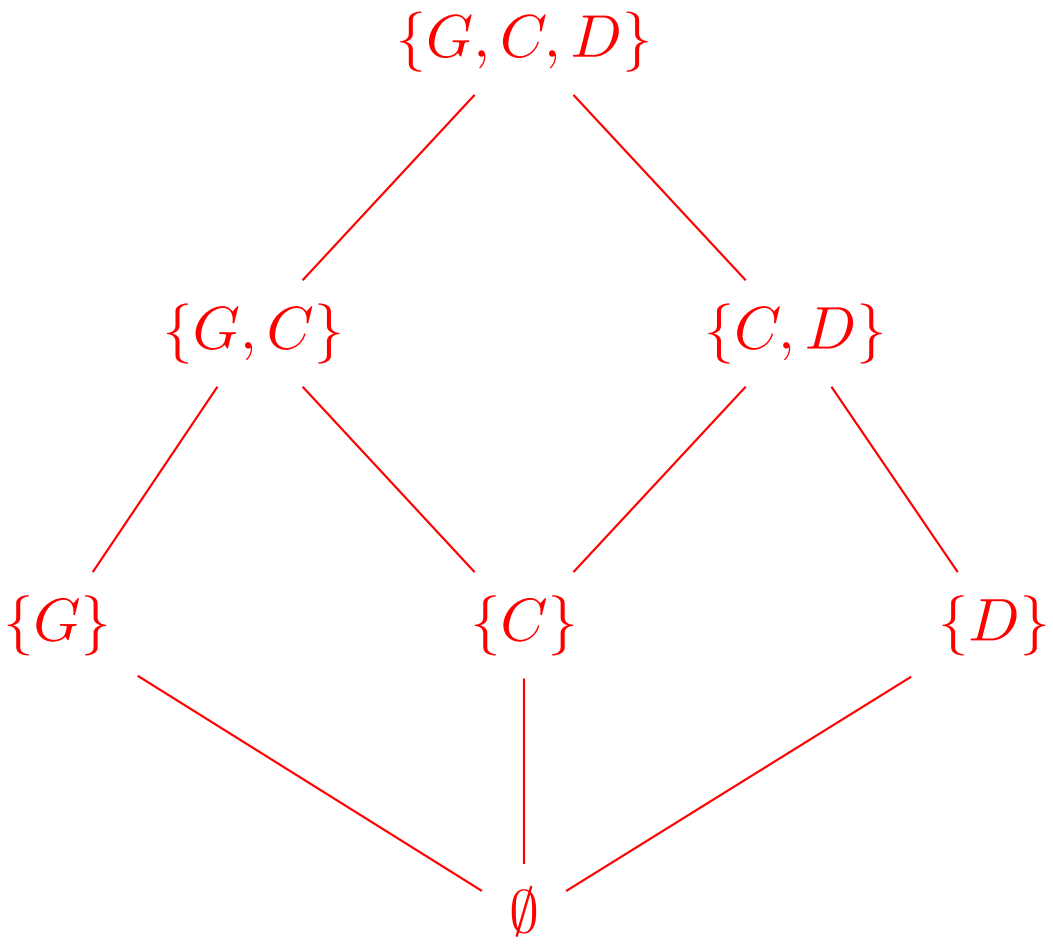
Définition 6

Une coalition $C \in \mathcal{P}(\mathcal{V})$ est dite *réaliste* si

$$\forall (i,j) \in C^2, \forall k \in \mathcal{V}, i \preccurlyeq k \preccurlyeq j \implies k \in C$$

Notons $\mathcal{R}$ l'ensemble des coalitions réalistes et $\mathcal{R}_i$ celles contenant le votant i.

$\mathcal{V} = \{G, C, D\}$ avec $G \preccurlyeq C \preccurlyeq D$



Treillis $(\mathcal{R}, \subseteq)$

Décidabilité réaliste

$$d_i^r = |\{C \in \mathcal{G} \cap \mathcal{R}_i \mid C \setminus \{i\} \notin \mathcal{G}\}|$$

Décidabilité réaliste

$$d_i^r = |\{C \in \mathcal{G} \cap \mathcal{R}_i \mid C \setminus \{i\} \notin \mathcal{G}\}|$$

$$B_i^r = \frac{d_i^r}{\sum_{j=1}^{N} d_j^r} \qquad B_i'^r = \frac{d_i^r}{|\mathcal{R}_i|}$$

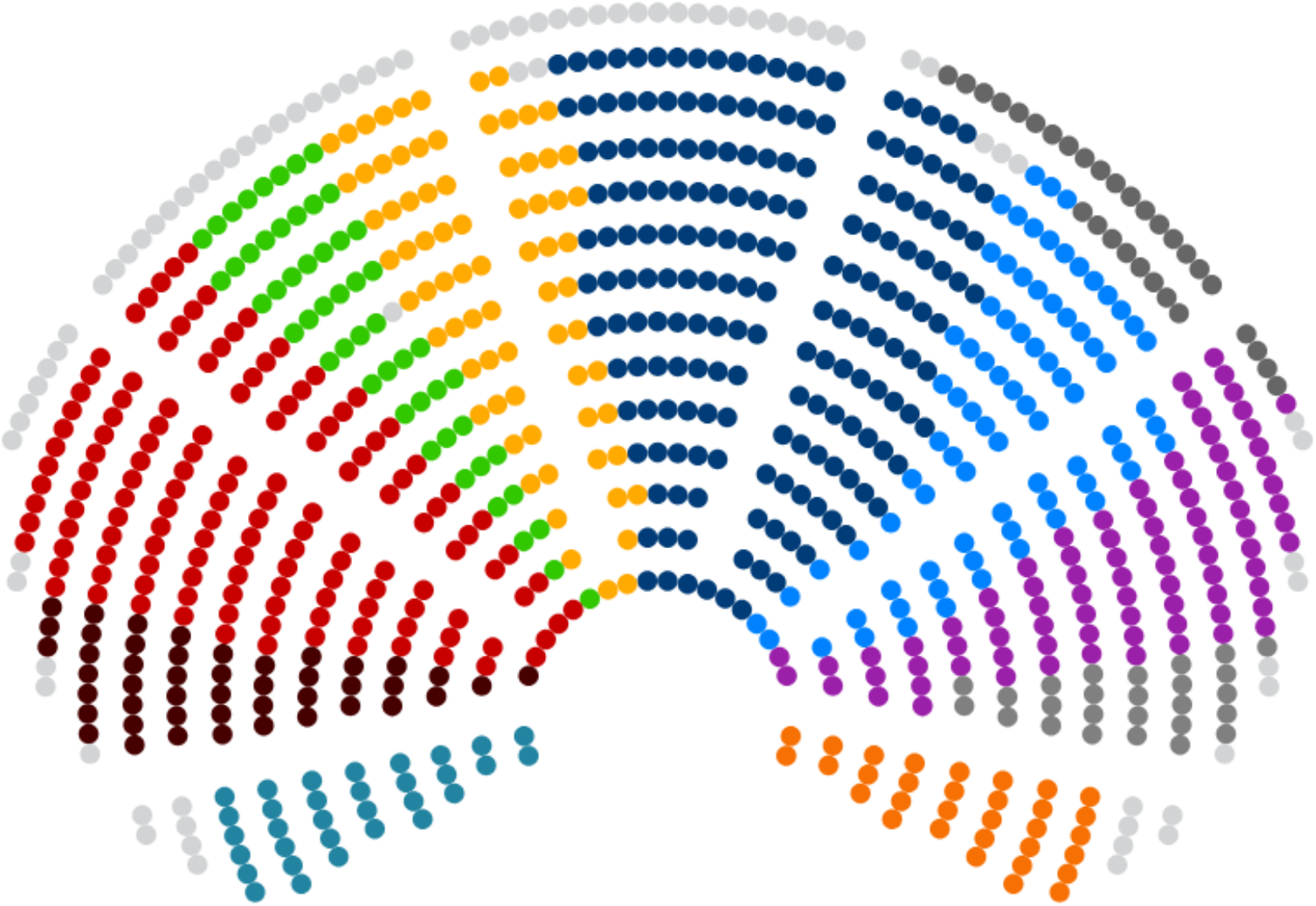
Considérons $(\mathcal{V}, \preccurlyeq) = ([\![1; N]\!], \leqslant)$.

$$\mathcal{R} = \{\emptyset\} \cup \left\{ [\![j; j']\!] \mid (j, j') \in \mathcal{V}^2, j \leqslant j' \right\}$$

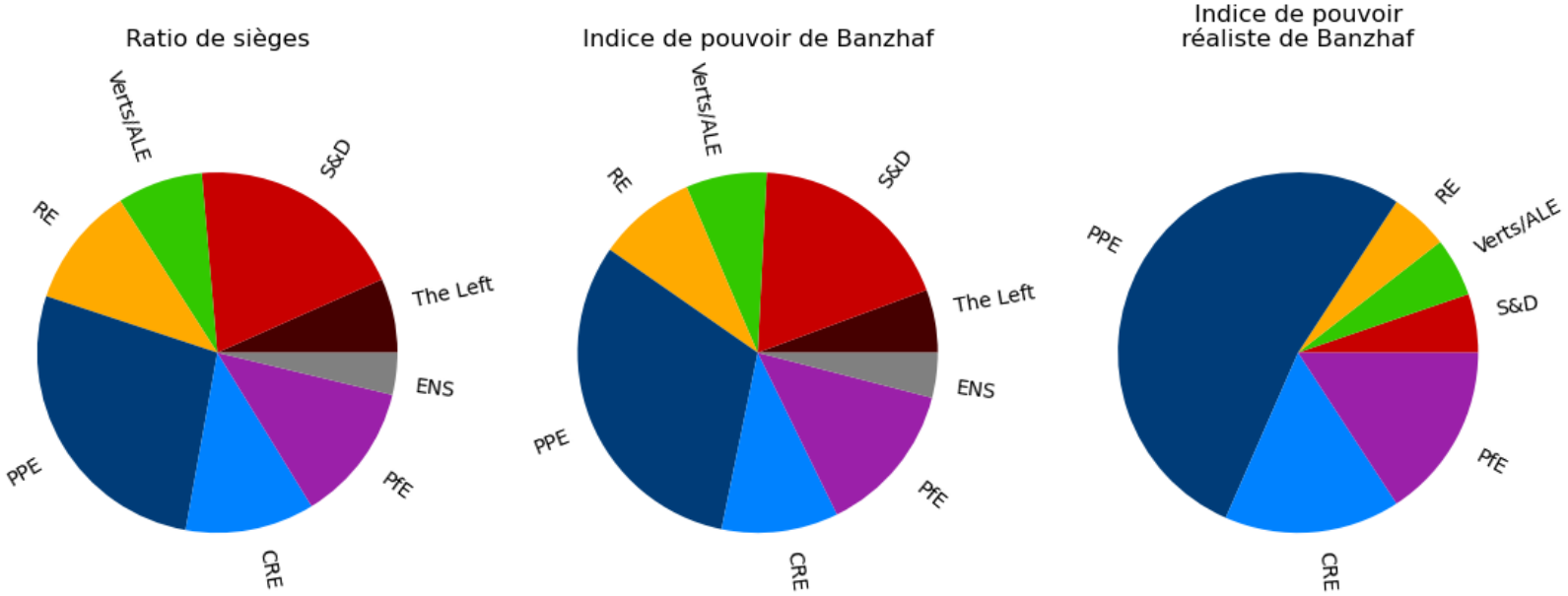
Considérons $(\mathcal{V}, \preccurlyeq) = ([\![1; N]\!], \leqslant)$.

$$\mathcal{R} = \{\emptyset\} \cup \left\{ [\![j; j']\!] \mid (j, j') \in \mathcal{V}^2, j \leqslant j' \right\}$$

Complexité de d_i^r : $\mathcal{O}(N^3)$



Pouvoir des groupes au Parlement Européen



Conclusion

- Penrose 62 est simple, transparent et facilement extensible à de nouveaux pays.
- J'ai adapté l'indice de Banzhaf au spectre gauche/droite pour comprendre les dynamiques parlementaires.

Programmes Python

- indice_pouvoir.py
- main.py
- parlement.py
- penrose62.py
- polynome.py
- realiste.py
- utils.py
- vote_pondere.py

indice_pouvoir.py

```python
# -*- coding: utf-8 -*-
"""
Calcul de l'indice de pouvoir
"""

import polynome as poly


def parties(ensemble):
    """Calcule l'ensemble des parties"""
    if len(ensemble) == 0:
        return [[]]
    if len(ensemble) == 1:
        return [[], ensemble]

    res = []

    e = ensemble[0]
    f = parties(ensemble[1:])
    for h in f:
        res.append(h.copy())
        res.append(h.copy())
        res[-1].append(e)

    return res


def indice_Banzhaf_naif(quota, poids):
    """Calcule l'indice de pouvoir de Banzhaf
    Algorithme naïf"""
    assert isinstance(quota, int)
    assert isinstance(poids, dict)
    votants = list(poids.keys())
```

```python

    gagnantes = []
    # itérer dans les coalitions
    for coalition in parties(votants):
        somme = 0
        for v in coalition:
            somme += poids[v]
        if somme >= quota:
            gagnantes.append((coalition, somme))

    decisif = dict.fromkeys(votants, 0)
    for coalition, somme in gagnantes:
        for v in coalition:
            if (somme - poids[v]) < quota:
                decisif[v] += 1

    somme_d = 0
    for v in votants:
        somme_d += decisif[v]

    if somme_d == 0:
        return {v: 0 for v in decisif}
    return {v: d/somme_d for v, d in decisif.items()}



def indice_Banzhaf_brut(quota, poids):
    """Calcule la décidabilité / l'indice de Banzhaf brut
    Algorithme avec les polynômes générateurs de Brams-Affuso"""
    assert isinstance(quota, int)
    assert isinstance(poids, dict)
    ibb = {} # indice de Banzhaf brut
    votants = poids.keys()
```

```python

    for v in votants:
        g = poly.Polynome([1]) # polynôme générateur
        for w in votants:
            if v != w:
                g *= poly.Polynome([1]) + poly.monome(poids[w])

        ibb[v] = 0
        for i in range(max(quota - poids[v], 0), min(quota, g.degre + 1)):
            ibb[v] += g.coef[i]

    return ibb



def indice_Banzhaf(quota, poids):
    """Calcule l'indice de Banzhaf normalisé
    Algorithme de Brams-Affuso avec les séries génératrices"""
    assert isinstance(quota, int)
    assert isinstance(poids, dict)

    # indice de Banzhaf brut
    ibb = indice_Banzhaf_brut(quota, poids)
    votants = poids.keys()

    somme_ibb = 0
    for v in votants:
        somme_ibb += ibb[v]

    if somme_ibb == 0:
        return {v: 0 for v in ibb}
    return {v: i/somme_ibb for v, i in ibb.items()}

```

indice_pouvoir.py

```python

def indice_Banzhaf_absolu(quota, poids):
    """Calcule l'indice de Banzhaf absolu
    Algorithme de Brams-Affuso avec les séries génératrices"""
    assert isinstance(quota, int)
    assert isinstance(poids, dict)
    ibb = indice_Banzhaf_brut(quota, poids) # indice de Banzhaf brut
    nb_votants = 2**(len(poids)-1)

    if nb_votants == 0:
        return {v: 0 for v in ibb}
    return {v: i/nb_votants for v, i in ibb.items()}


def sensibilite(quota, poids):
    """"Calcule la sensibilité ou pouvoir d'une collectivité à agir"""
    return sum(indice_Banzhaf_brut(quota, poids).values()) / 2**(len(poids)-1)

```

main.py

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Fichier principal
"""

import time

import parlement as prl
import vote_pondere as vp
import penrose62
import indice_pouvoir as ip


def main():
    start = time.process_time()

    # Décommenter une line pour effectuer un calcul
    # prl.indice_parlement(prl.parlement_UE["pays"])
    # prl.indice_parlement(prl.parlement_UE["groupe"], reel=True)
    # print("différence maximale absolue/relative", vp.poids_parlement_UE())
    # penrose62.EU_Penrose(pays_candidats=True, coef=6, qf=0.6, passer_sigma=True)
    # penrose62.EU_Penrose(qf=0.62)

    end = time.process_time()

    print(f"Temps de calcul : {end - start}s")

if __name__ == "__main__":
    main()
```

parlement.py

```python
# -*- coding: utf-8 -*-
"""
Utilitaire et données de différents parlements
"""


import math
import matplotlib.pyplot as plt

import utils
import indice_pouvoir as ip
import realiste


def indice_parlement(parlement, quota_relatif=1/2, verbose=False, plotted=True, reel=False, ndigits=1, normalise=True):
    """Calcule l'indice de pouvoir des différents groupes dans une assemblée
    Le quota s'exprime en pourcentage
    Mettre verbose à true pour afficher les résultats détaillés
    reel indique si les indices doivent réalistes ou non (par défaut non)"""
    assert isinstance(parlement, Parlement)


    total = parlement.total
    quota = math.ceil(total*quota_relatif)

    ratio = {pays: s/total for pays, s in parlement.sieges.items()}
    if normalise:
        pouvoir = ip.indice_Banzhaf(quota, parlement.sieges)
    else:
        pouvoir = ip.indice_Banzhaf_absolu(quota, parlement.sieges)
    if reel:
        if normalise:
```

```
            pouvoir_realiste = realiste.indice_Banzhaf(quota, parlement.sieges)
        else:
            pouvoir_realiste = realiste.indice_Banzhaf_absolu(quota, parlement.sieges)

    difference = {}
    difference_realiste = {}
    for groupe in parlement.sieges.keys():
        difference[groupe] = (pouvoir[groupe] - ratio[groupe]) / ratio[groupe]
        if reel:
            difference_realiste[groupe] = (pouvoir_realiste[groupe]  - ratio[groupe]) /
            ↪  ratio[groupe]

    if verbose:
        utils.print_dictionnaire(ratio, "ratio")
        utils.print_dictionnaire(difference, "différence")
        if normalise:
            utils.print_dictionnaire(pouvoir, "pouvoir")
        else:
            utils.print_dictionnaire(pouvoir, "pouvoir absolu")
        if reel:
            if normalise:
                utils.print_dictionnaire(pouvoir_realiste, "pouvoir réaliste")
            else:
                utils.print_dictionnaire(pouvoir_realiste, "pouvoir réaliste absolu")
            utils.print_dictionnaire(difference_realiste, "différence réaliste")
    print("sensibilité : ", ip.sensibilite(quota, parlement.sieges))

    if plotted:
        data_pouvoir = [
            (ratio, "Ratio de sièges"),
            (pouvoir, "Indice de pouvoir de Banzhaf")
        ]
        if reel:
```

parlement.py

```python
            data_pouvoir.append((pouvoir_realiste, "Indice de pouvoir\nréaliste de Banzhaf"))
        utils.plot_pie(data_pouvoir, couleurs=parlement.couleurs)

        utils.plot_bar(difference.keys(), difference.values(), "", couleurs=parlement.couleurs, ndigits=ndigits) # "Écart relatif entre le pouvoir et la représentation"

        if reel:
            utils.plot_bar(difference_realiste.keys(), difference_realiste.values(), "Écart relatif entre le pouvoir réaliste et la représentation", couleurs=parlement.couleurs, ndigits=ndigits)

    return pouvoir


class Parlement:
    def __init__(self, _sieges, _source, _couleurs=None):
        self.sieges = _sieges
        self.source = _source
        self.couleurs = _couleurs

    @property
    def total(self):
        """Nombre total de parlementaires"""
        return sum(self.sieges.values())


parlement_UE = {
    "pays": Parlement( # total = 720
        {
            "Allemagne": 96,
            "France": 81,
            "Italie": 76,
            "Espagne": 60,
```

```
            "Pologne": 53,
            "Roumanie": 33,
            "Pays-Bas": 31,
            "Belgique": 22,
            "République\ntchèque": 21,
            "Grèce": 21,
            "Hongrie": 21,
            "Portugal": 21,
            "Suède": 21,
            "Autriche": 20,
            "Bulgarie": 17,
            "Danemark": 15,
            "Finlande": 15,
            "Slovaquie": 15,
            "Irlande": 14,
            "Croatie": 12,
            "Lituanie": 11,
            "Lettonie": 9,
            "Slovénie": 9,
            "Estonie": 7,
            "Chypre": 6,
            "Luxembourg": 6,
            "Malte": 6
        },
        "https://www.europarl.europa.eu/meps/fr/search/table"
        ),
    "groupe": Parlement(
        {
            "The Left": 46,
            "S&D": 136,
            "Verts/ALE": 53,
            "RE": 75,
            "PPE": 188,
```

parlement.py

```
            "CRE": 80,
            "PfE": 86,
            "ENS": 26
        },
        "https://www.europarl.europa.eu/meps/fr/search/chamber",
        {
            "The Left": "#460000",
            "S&D": "#c80000",
            "Verts/ALE": "#32c800",
            "RE": "#ffaa00",
            "PPE": "#003c78",
            "CRE": "#0082ff",
            "PfE": "#9b20a9",
            "ENS": "#808080"
        }
        )
    }
```

penrose62.py

```python
# -*- coding: utf-8 -*-
"""
Teste le système de vote Penrose-62
"""


import math
import matplotlib.pyplot as plt
import numpy as np

import utils
import parlement as prl
import indice_pouvoir as ip


def EU_Penrose(coef=8, pays_candidats=False, passer_sigma=False, qf=0.62, comparer62=False):
    """Calcule les indices de pouvour dans Penrose-62"""
    # Source: https://en.wikipedia.org/wiki/List_of_European_Union_member_states_by_population
    population = { # en million
        "Allemagne": 83.4,
        "France": 68.4,
        "Italie": 59.0,
        "Espagne": 48.6,
        "Pologne": 36.6,
        "Roumanie": 19.1,
        "Pays-Bas": 17.9,
        "Belgique": 11.8,
        "République\ntchèque": 10.9,
        "Portugal": 10.6,
        "Suède": 10.6,
        "Grèce": 10.4,
        "Hongrie": 9.6,
        "Autriche": 9.2,
```

```
        "Bulgarie": 6.4,
        "Danemark": 6.0,
        "Finlande": 5.6,
        "Slovaquie": 5.4,
        "Irlande": 5.3,
        "Croatie": 3.9,
        "Lituanie": 2.9,
        "Slovénie": 2.1,
        "Lettonie": 1.9,
        "Estonie": 1.4,
        "Chypre": 0.9,
        "Luxembourg": 0.7,
        "Malte": 0.6
    }

    if pays_candidats:
        # Sources pour les pays :
        # https://fr.wikipedia.org/wiki/Union_europ%C3%A9enne
        # https://european-union.europa.eu/principles-countries-history/eu-enlargement_en
        # Source pour le nombre d'hbitants :
        # Wikipédia
        population.update({
            "Albanie": 2.4,
            "Bosnie-\nHerzégovine": 3.2,
            "Géorgie": 4.0,
            "Macédoine\ndu Nord": 1.8,
            "Moldavie": 2.7,
            "Monténégro": 0.6,
            "Serbie": 6.7,
            "Turquie": 85.4,
            "Ukraine": 38.7
            })

```

penrose62.py

```python
    total_pop = sum(population.values())
    pourcent_pop = {k: v/total_pop for k, v in population.items()}
    utils.print_dictionnaire(pourcent_pop, "Part de la population européenne par pays")

    sqrt_pop = prl.Parlement({}, "Penrose")
    for p in population:
        sqrt_pop.sieges[p] = round(math.sqrt(population[p]) * coef)
    sum_sqrt = sum(sqrt_pop.sieges.values())
    pop_normalise = prl.Parlement({k: v / sum_sqrt for k, v in sqrt_pop.sieges.items()}, "Penrose")

    utils.print_dictionnaire(sqrt_pop.sieges, "Sièges au parlement par états", True)
    print("Quota : ", math.ceil(sum_sqrt*qf) )

    if not passer_sigma:
        quota_arr = [ i/100 for i in range(50, 71) ]
        pourvoir_arr = list( map(
                lambda i :
                    ( lambda d :
                        sum([
                            (pop_normalise.sieges[v] - d[v])**2
                        for v in d.keys() ])
                    )
                    ( ip.indice_Banzhaf( math.ceil(sum_sqrt * i), sqrt_pop.sieges) ),
                quota_arr
            ))
        print(pourvoir_arr)

        plt.plot(quota_arr, pourvoir_arr, marker='o')
        plt.xticks([0.5, 0.55, 0.6, 0.65, 0.7])
        plt.xlabel("quota relatif")
        plt.ylabel(r'$\sigma$   ', rotation=0).set_fontsize(15)
        plt.show()
```

penrose62.py

```python

    pouvoir = prl.indice_parlement(sqrt_pop, qf)

    proportionnalite = {}
    for p in population:
        proportionnalite[p] = pouvoir[p] / math.sqrt(population[p])

    utils.plot_bar(proportionnalite.keys(), proportionnalite.values(), "Rapport de l'indice de pouvoir par rapport la racine carrée de la population", ndigits=1)

    if comparer62:
        pouvoir62 = ip.indice_Banzhaf(math.ceil(sum_sqrt*0.62), sqrt_pop.sieges)

        y_pos = np.array(range(len(pop_normalise.sieges)))
        bar_size = 0.2

        plt.bar(y_pos-bar_size, pouvoir62.values(), color='b', width=bar_size)
        plt.bar(y_pos, pouvoir.values(), color='g', width=bar_size)
        plt.bar(y_pos+bar_size, pop_normalise.sieges.values(), color='r', width=bar_size)
        plt.xticks(y_pos, pop_normalise.sieges.keys(), rotation=60)

        plt.subplots_adjust(bottom=0.15)
        plt.get_current_fig_manager().full_screen_toggle()
        plt.show()
```

polynome.py

```python
# -*- coding: utf-8 -*-
"""
Implémentation des polynômes
"""

import math

class Polynome:
    """Classe modélisant un polynôme sous forme de liste"""

    def __init__(self, coef):
        """Initialise le polynôme avec la liste de ses coefficents"""
        # liste des coefficents dans l'ordre croissant des puissances
        # Theta(len coef)
        self.coef = coef.copy()

    def est_nul(self):
        """Teste si le polynôme est le polynôme nul"""
        for c in self.coef:
            if c != 0:
                return False
        return True

    def coefficents_nuls(self):
        """Enlève les coefficents nuls"""
        while len(self.coef) != 0 and self.coef[-1] == 0:
            del self.coef[-1]

    @property
    def degre(self):
        """Degré du polynôme"""
        self.coefficents_nuls()
        if self.est_nul():
```

polynome.py

```
            return - math.inf
        return len(self.coef) - 1

    def __str__(self):
        """Retourne un représentation du polynôme"""
        if self.est_nul():
            return "0"
        res = str(self.coef[0])
        for i in range(1, self.degre + 1):
            res += f" + {str(self.coef[i])}*X^{i}"
        return res

    def copier(self):
        """Copie le polynôme"""
        # Theta(deg self)
        return Polynome(self.coef.copy())

    def __iadd__(self, p):
        """Ajoute p au polynôme"""
        # Theta(deg P)
        assert isinstance(p, Polynome)
        if p.est_nul():
            return self
        for i in range(p.degre + 1):
            if i < len(self.coef):
                self.coef[i] += p.coef[i]
            else:
                self.coef.append(p.coef[i])
        return self

    def __add__(self, p):
        """Additionne deux polynômes"""
        # Theta(min(deg Self, deg P))
```

```
        if self.degre >= p.degre:
            r = self.copier()
            s = p
        else:
            r = p.copier()
            s = self

        r += s
        return r

    def __mul__(self, p):
        """Multiplie deux polynômes"""
        if isinstance(p, (float,int)):
            # Theta(deg Self)
            res = self.copier()
            for i in range(self.degre + 1):
                res.coef[i] *= p

        elif isinstance(p, Polynome):
            if self.degre < p.degre:
                return p * self
            # Theta( (deg P) (deg Self + deg P) )
            res = Polynome([])
            if not p.est_nul():
                for i in range(p.degre + 1):
                    # Theta(deg P + i)
                    pp = self.copier()
                    pp *= p.coef[i]
                    for _ in range(i):
                        pp.coef.insert(0, 0)
                    res += pp

        else:
```

polynome.py

```
            raise TypeError("p doit être un entier, un flottant ou un polynôme.")

        return res

    def __imul__(self, p):
        """"Multiplie ce polynôme par p"""
        # Theta(deg P * (deg Self + deg P))
        self = self * p
        return self


def monome(n):
    """Crée un monôme de degré n."""
    # Theta(n)
    return Polynome([0] * n + [1])
```

realiste.py

```python
# -*- coding: utf-8 -*-
"""
Calcul de l'indice de pouvoir réaliste
"""

def coalitions_realistes(l):
    """Calcule l'ensemble des coalitions réalistes de l triée"""
    n = len(l)
    res = [[]]

    for i in range(n+1):
        for j in range(i+1, n+1):
            res.append(l[i:j])

    return res

def nb_realistes(n):
    """Calcule le nombre de coalitions réalistes"""
    return 1 + n*(n+1)/2


def indice_Banzhaf_brut(quota, poids):
    """Calcule l'indice de pouvoir réaliste de Banzhaf
    Algorithme naïf"""
    assert isinstance(quota, int)
    assert isinstance(poids, dict)
    votants = list(poids.keys())

    gagnantes = []
    # itérer dans les coalitions réalistes
    for coalition in coalitions_realistes(votants):
        somme = 0
        for v in coalition:
```

realiste.py

```
            somme += poids[v]
        if somme >= quota:
            gagnantes.append((coalition, somme))

    decisif = dict.fromkeys(votants, 0)
    for coalition, somme in gagnantes:
        for v in coalition:
            if (somme - poids[v]) < quota:
                decisif[v] += 1
    return decisif


def indice_Banzhaf(quota, poids):
    """Calcule l'indice de Banzhaf réaliste"""
    ibb = indice_Banzhaf_brut(quota, poids)
    somme_d = sum(ibb.values())

    if somme_d == 0:
        return {v: 0 for v in ibb}
    return {v: d/somme_d for v, d in ibb.items()}


def indice_Banzhaf_absolu(quota, poids):
    """Calcule l'indice de Banzhaf absolu réaliste"""
    assert isinstance(quota, int)
    assert isinstance(poids, dict)
    ibb = indice_Banzhaf_brut(quota, poids) # indice de Banzhaf brut
    votants = list(poids.keys)
    nb_votants = len(poids)

    if nb_votants == 0:
        return {v: 0 for v in ibb.keys()}
    return {v: i/( votants.index(v) * (nb_votants - votants.index(v) + 1) ) for v, i in
    ↪  ibb.items()}
```

utils.py

```python
# -*- coding: utf-8 -*-
"""
Toutes les petites fonctions utiles
"""

import math
import matplotlib.pyplot as plt


def print_dictionnaire(d, nom=None, print_total=False):
    """Affiche un dictionnaire"""

    if print_total:
        total = 0

    if nom is not None:
        print(nom, " : ", end="")
    print("{")
    for clef, valeur in d.items():
        print(f"\t{clef} : {valeur}")
        if print_total:
            total += valeur
    if print_total:
        print(f"\tTOTAL : {total}")
    print("}")

def plot_bar(x, y, titre, couleurs=None, ndigits=1):
    assert len(x) == len(y), "x et y doivent être de la même taille"

    y_pos = range(len(x))
    if couleurs == None:
        couleurs_pos = ["deepskyblue"]
        rot = 70
```

```
    else:
        couleurs_pos = list(couleurs.values())
        rot = 0

    rectarr = plt.bar(y_pos, y, color=couleurs_pos)
    for rect in rectarr:
        height = rect.get_height()
        if (height < 0) == (couleurs==None):
            va = "bottom"
        else:
            va = "top"

        plt.text(
            rect.get_x() + rect.get_width()/2,
            height,
            str(round(height*100, ndigits)) + "%",
            ha="center",
            va=va,
            rotation=rot
            ).set_fontsize(9)
    plt.xticks(y_pos, x, rotation=60)
    plt.title(titre)
    plt.subplots_adjust(bottom=0.15)
    plt.get_current_fig_manager().full_screen_toggle()

    # ax = plt.gca()
    # ax.set_ylim([-0.1, 0.1])

    plt.show()


def plot_pie(data, couleurs=None):
    """Affiche des diagrammes circulaires
```

utils.py

```
    un par élément de data qui sont de la forme (dictionnaire, titre)
    couleurs donne les couleurs des données (optionel)"""
    n = len(data)
    fig, axarr = plt.subplots(1, n)

    for i in range(n):
        if couleurs == None:
            c = None
        else:
            c = [] # couleurs
        dwz = {} # data without zero
        for k, v in data[i][0].items():
            if v != 0:
                dwz[k] = v
                if couleurs != None:
                    c.append(couleurs[k])

        axarr[i].pie(dwz.values(), labels=dwz.keys(), rotatelabels=True, colors=c)
        axarr[i].set_title(data[i][1], pad=40)

    # fig.tight_layout(pad=-2)
    # fig.set_figwidth(5)
    plt.get_current_fig_manager().full_screen_toggle()
    plt.show()
```

vote_pondere.py

```python
# -*- coding: utf-8 -*-
"""
Crée un vote pondéré pour des indices de Banzhaf et un quota donnés
"""


import math
import matplotlib.pyplot as plt

import utils
import indice_pouvoir as ip


def arrondi_vote(vote):
    """Arrondi les poids du système de vote"""
    res = {}
    for v, p in vote.items():
        res[v] = round(p)
    return res


def vote_pondere(quota, pouvoir, iteration=40):
    """Calcule un vote pondéré
    dont les indices de pouvoir de Banzhaf sont ceux donnés en entrée
    Algorithme de Leech 2002"""
    assert isinstance(pouvoir, dict)

    poids = pouvoir.copy()

    for i in range(1, iteration + 1):
        tmp = ip.indice_Banzhaf_absolu(quota, arrondi_vote(poids))
        alpha = 0.83 * quota / ( math.log(i) + 1 )
        for v in pouvoir:
```

vote_pondere.py

```
            poids[v] += alpha * (pouvoir[v] - tmp[v])
            poids[v] = max(1, poids[v])

    poids = arrondi_vote(poids)

    return poids


def poids_parlement(quota, quantite, verbose=False, plotted=True):
    """Calcule le poids des différents groupes dans une assemblée
    de manière à ce que l'indice de Banzhaf soit le plus proche possible de la quantité
    Le quota s'exprime en pourcentage
    Mettre verbose à true pour afficher les résultats détaillés"""

    total = sum(quantite.values())
    ratio_quantite = {pays: s/total for pays, s in quantite.items()}
    poids = vote_pondere(quota, ratio_quantite)
    ratio_poids = {pays: s/total for pays, s in poids.items()}
    pouvoir = ip.indice_Banzhaf(quota, poids)

    difference = {}
    difference_rel = {}
    for groupe in quantite.keys():
        difference[groupe] = pouvoir[groupe] - ratio_quantite[groupe]
        difference_rel[groupe] = difference[groupe] / ratio_quantite[groupe]

    if verbose:
        utils.print_dictionnaire(ratio_quantite, "ratio")
        utils.print_dictionnaire(pouvoir, "pouvoir réel")
        utils.print_dictionnaire(difference, "écart")
    utils.print_dictionnaire(poids, "Sièges au parlement par états", True)
    print(f"Sensibilité : {ip.sensibilite(quota, poids)}")

```

vote_pondere.py

```python
    if plotted:
        utils.plot_pie([
            (ratio_poids, "Sièges"),
            (ratio_quantite, "Population européenne"),
            (pouvoir, "Indice de Banzhaf")
            ])

        utils.plot_bar(difference.keys(), difference.values(), "") # "Écart entre le pouvoir et la représentation")

        utils.plot_bar(difference_rel.keys(), difference_rel.values(), "") # "Écart relatif entre le pouvoir et la représentation")

    # calcul des écarts maxima
    max_abs = 0 # absolu
    max_rel = 0 # relatif
    for groupe in quantite.keys():
        if abs(difference[groupe]) > abs(max_abs):
            max_abs = difference[groupe]
        if abs(difference_rel[groupe]) > abs(max_rel):
            max_rel = difference_rel[groupe]

    return (max_abs, max_rel)


def poids_parlement_UE(quota = 400):
    """Source:
    https://en.wikipedia.org/wiki/List_of_European_Union_member_states_by_population"""
    population = { # en million
        "Allemagne": 83.4,
        "France": 68.4,
        "Italie": 59.0,
        "Espagne": 48.6,
```

vote_pondere.py

```
        "Pologne": 36.6,
        "Roumanie": 19.1,
        "Pays-Bas": 17.9,
        "Belgique": 11.8,
        "République\ntchèque": 10.9,
        "Portugal": 10.6,
        "Suède": 10.6,
        "Grèce": 10.4,
        "Hongrie": 9.6,
        "Autriche": 9.2,
        "Bulgarie": 6.4,
        "Danemark": 6.0,
        "Finlande": 5.6,
        "Slovaquie": 5.4,
        "Irlande": 5.3,
        "Croatie": 3.9,
        "Lituanie": 2.9,
        "Slovénie": 2.1,
        "Lettonie": 1.9,
        "Estonie": 1.4,
        "Chypre": 0.9,
        "Luxembourg": 0.7,
        "Malte": 0.6
    }

    sqrt_population = {}
    for p in population:
        sqrt_population[p] = math.sqrt(population[p])

    print(f"Quota : {quota}")
    return poids_parlement(quota, sqrt_population)
```